PRAKTIKUM SISTEM OPERASI

MODUL VI

PROSES SEKUENSIAL DAN KONKUREN

Disusun Oleh :

Alya Rabani / 2311104076

SE-07-02

Dosen Pengampu :

Yudha Islami Sulistya, S.Kom., M.Cs.

PROGRAM STUDI S1 SOFTWARE ENGINEERING

FAKULTAS INFORMATIKA

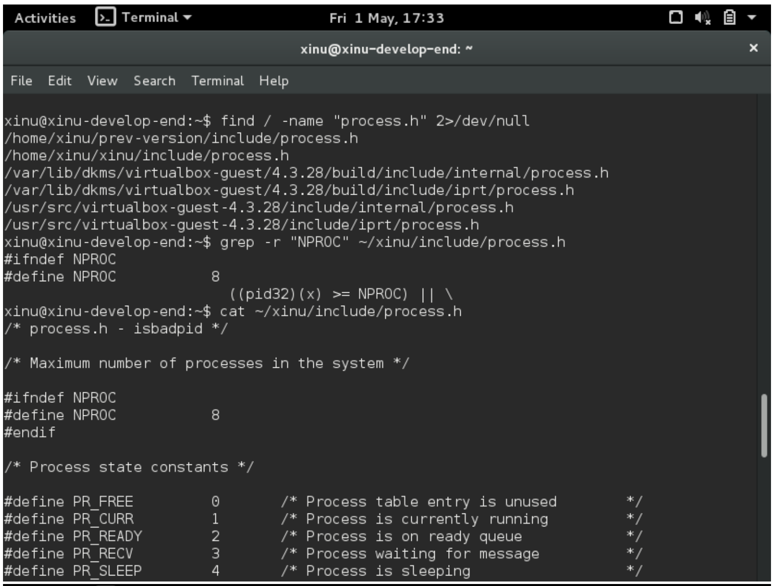
TELKOM UNIVERSITY PURWOKERTO

2025

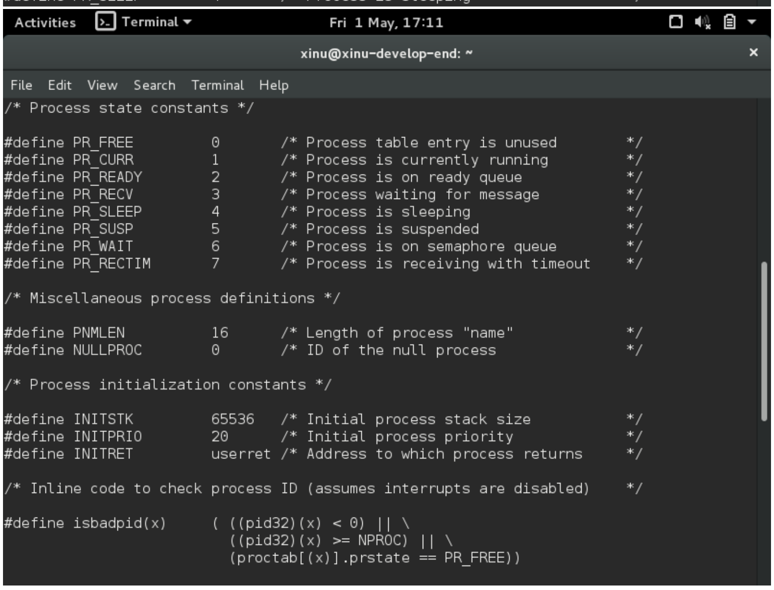
JURNAL

1. Selain hardware (memory), batasan maksimal proses dapat ditentukan dengan secara software. Pada Linux maksimal proses adalah 4194303 proses (64 bit) dan 32767 proses (32 bit) dapat dilihat melalui perintah $cat /proc/sys/kernel/pid_max

   - File process.h

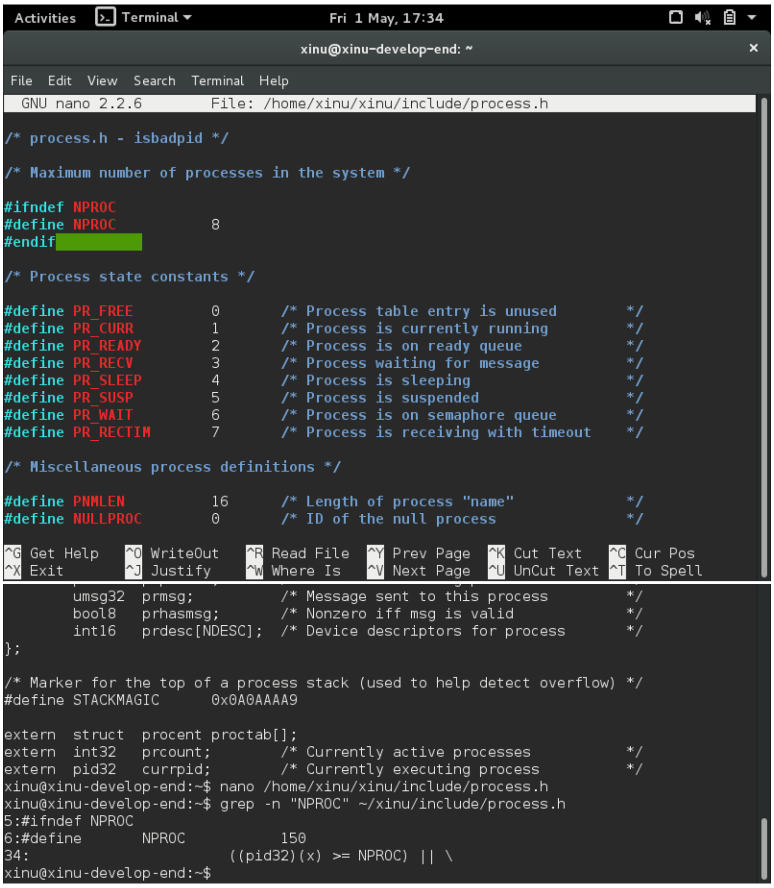


```
Activities   Terminal ▾          Fri 1 May, 17:33
xinu@xinu-develop-end: ~
File  Edit  View  Search  Terminal  Help

xinu@xinu-develop-end:~$ find / -name "process.h" 2>/dev/null
/home/xinu/prev-version/include/process.h
/home/xinu/xinu/include/process.h
/var/lib/dkms/virtualbox-guest/4.3.28/build/include/internal/process.h
/var/lib/dkms/virtualbox-guest/4.3.28/build/include/iprt/process.h
/usr/src/virtualbox-guest-4.3.28/include/internal/process.h
/usr/src/virtualbox-guest-4.3.28/include/iprt/process.h
xinu@xinu-develop-end:~$ grep -r "NPROC" ~/xinu/include/process.h
#ifndef NPROC
#define NPROC           8
                          ((pid32)(x) >= NPROC) || \
xinu@xinu-develop-end:~$ cat ~/xinu/include/process.h
/* process.h - isbadpid */

/* Maximum number of processes in the system */

#ifndef NPROC
#define NPROC           8
#endif

/* Process state constants */

#define PR_FREE         0       /* Process table entry is unused       */
#define PR_CURR         1       /* Process is currently running        */
#define PR_READY        2       /* Process is on ready queue           */
#define PR_RECV         3       /* Process waiting for message         */
#define PR_SLEEP        4       /* Process is sleeping                 */
```



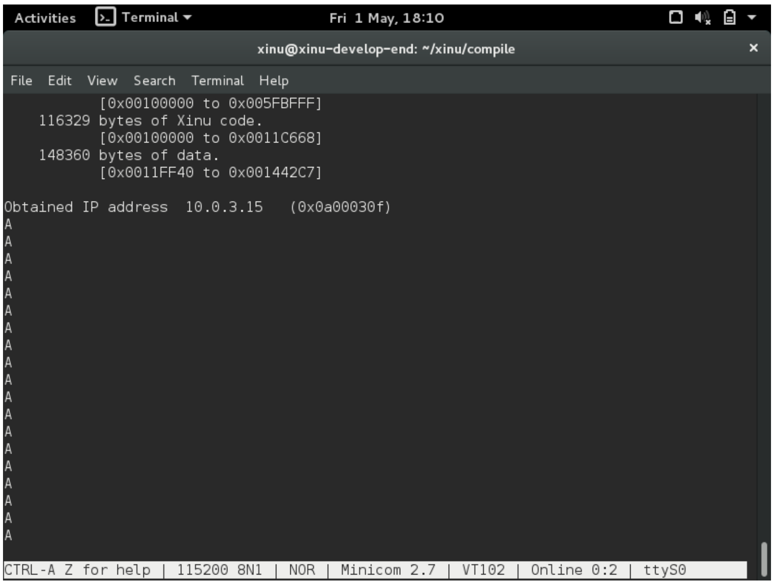
```
Activities   Terminal ▾          Fri 1 May, 17:11
xinu@xinu-develop-end: ~
File  Edit  View  Search  Terminal  Help
/* Process state constants */

#define PR_FREE         0       /* Process table entry is unused       */
#define PR_CURR         1       /* Process is currently running        */
#define PR_READY        2       /* Process is on ready queue           */
#define PR_RECV         3       /* Process waiting for message         */
#define PR_SLEEP        4       /* Process is sleeping                 */
#define PR_SUSP         5       /* Process is suspended                */
#define PR_WAIT         6       /* Process is on semaphore queue       */
#define PR_RECTIM       7       /* Process is receiving with timeout   */

/* Miscellaneous process definitions */

#define PNMLEN          16      /* Length of process "name"            */
#define NULLPROC        0       /* ID of the null process              */

/* Process initialization constants */

#define INITSTK         65536   /* Initial process stack size          */
#define INITPRIO        20      /* Initial process priority            */
#define INITRET         userret /* Address to which process returns    */

/* Inline code to check process ID (assumes interrupts are disabled)   */

#define isbadpid(x)     ( ((pid32)(x) < 0) || \
                          ((pid32)(x) >= NPROC) || \
                          (proctab[(x)].prstate == PR_FREE))
```

- Mengubah NPROC dari 8 ke 150



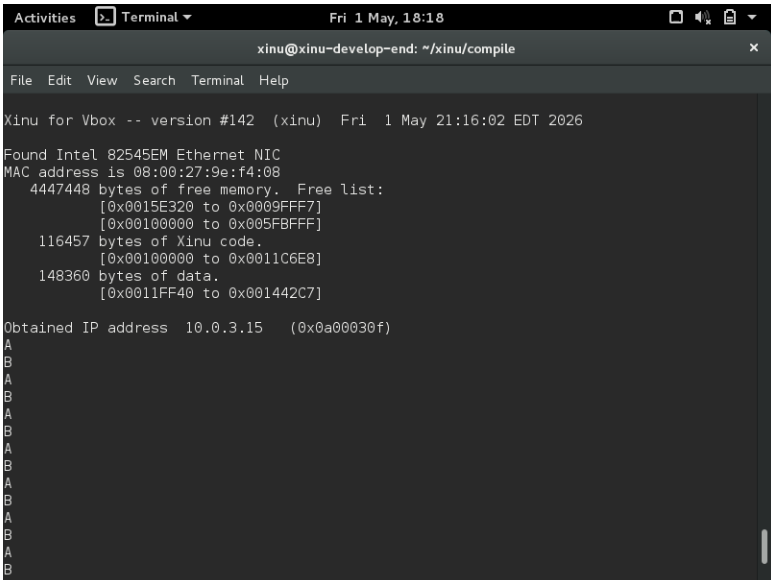
2. Jalankan kode main_ex1!

```c
/* main.c - main*/

#include <xinu.h>
void sndA(void);
void sndB(void);

process main(void)
{
        sndA();
        sndB();
        return OK;
}

void sndA(void)
{
        while(1){
        printf("A\n");
        sleepms(1000);
        }
}

void sndB(void)
{
        while(1){
        printf("B\n");
        sleepms(1000);
        }
}
```



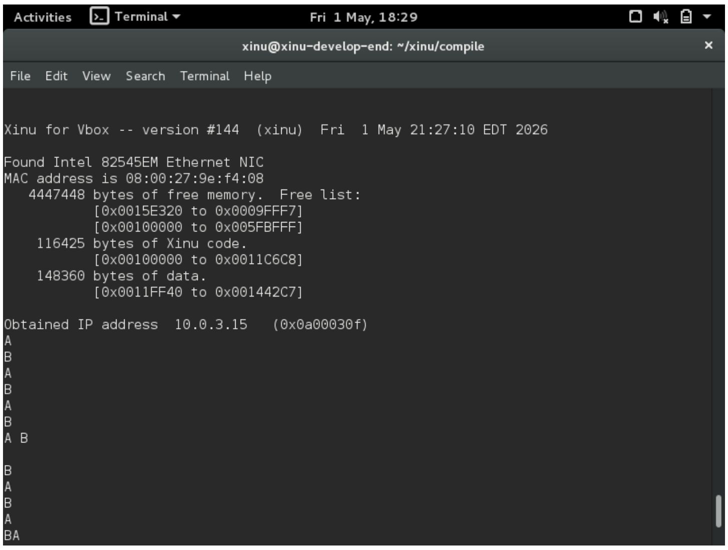
3. Jalankan kode main_ex2!

```c
/* main.c - main*/

#include <xinu.h>
void sndA(void);
void sndB(void);

process main(void)
{
        resume(create(sndA, 1024, 20, "process A", 0));
        resume(create(sndB, 1024, 20, "process B", 0));
        return OK;
}

void sndA(void)
{
        while(1){
        printf("A\n");
        sleepms(1000);
        }
}

void sndB(void)
{
        while(1){
        printf("B\n");
        sleepms(1000);
        }
}
```



4. Jalankan kode main_ex3!

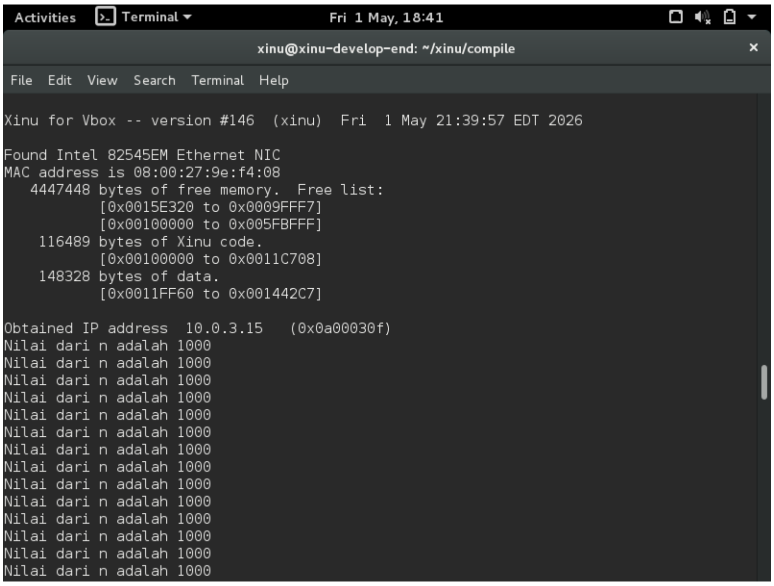
```
/* main.c - main*/

#include <xinu.h>
void sndChar(char);

process main(void)
{
        resume(create(sndChar, 1024, 20, "Send A", 1, 'A'));
        resume(create(sndChar, 1024, 20, "Send B", 1, 'B'));
        return OK;
}

void sndChar(char c)
{
        while(1){
        printf("%c \n", c);
        sleepms(1000);
        }
}
```



5. Buatlah 2 proses produser dan konsumer. Produser memproduksi angka integer dari 1-1000. Konsumer mengkonsumsi integer yang diproduksi oleh produser dan menampilkannya!

(Gunakan variabel global bertipe int32 bernama n yang digunakan secara bersama oleh kedua proses)

```c
/* main.c - main*/

#include <xinu.h>

int32 n = 0;

void produser(void) {
        int32 i;
        for (i = 1; i <= 1000; i++) {
                n++;
        }
}

void konsumer(void) {
        int32 i;
        for (i = 1; i <= 1000; i++) {
                printf("Nilai dari n adalah %d\n", n);
        }
}

process main(void) {
        resume(create(produser, 1024, 20, "producer", 0));
        resume(create(konsumer, 1024, 20, "consumer", 0));
        return OK;
}
```



Hasil program menunjukkan bahwa nilai n selalu 1000 karena proses produser menyelesaikan seluruh iterasi terlebih dahulu sebelum proses konsumer dijalankan. Hal ini terjadi karena tidak adanya delay atau mekanisme pembagian waktu yang memaksa kedua proses berjalan secara bergantian. Meskipun demikian, program ini tetap memiliki potensi race condition karena variabel n diakses secara bersama tanpa sinkronisasi. Jika penjadwalan proses berbeda, maka hasil yang ditampilkan dapat menjadi tidak terurut atau tidak konsisten.